1. Implement a Quadratic Equation Solver

Problem Statement: Write a C++ program that solves a quadratic equation of the form:

$$ax^2 + bx + c = 0$$

Create a class QuadraticEquation with attributes a, b, and c.

Implement a method to calculate the discriminant:

$$D = b^2 - 4ac$$

Implement a method to find the roots based on the discriminant.

```
# include <iostream>
# include <cmath>

class QuadraticEquation {
private:
    double a, b, c;
public:
    QuadraticEquation(double A, double B, double C) : a(A), b(B), c(C) {}

    double getDiscriminant() const {
        return b * b - 4 * a * c;
    }

    void findRoots() const {
        double D = getDiscriminant();
        if (D > 0) {
            double root1 = (-b + std::sqrt(D)) / (2 * a);
            double root2 = (-b - std::sqrt(D)) / (2 * a);
            std::cout << "Roots are real and different: " << root1 << ", " << root2 << std::endl;
        } else if (D == 0) {
            double root = -b / (2 * a);
            std::cout << "Roots are real and equal: " << root << std::endl;
        } else {
            std::cout << "No real roots, complex roots exist." << std::endl;
        }
    }
};

int main() {
    double a, b, c;
    std::cout << "Enter coefficients a, b, c: ";
    std::cin >> a >> b >> c;
```

```
    QuadraticEquation equation(a, b, c);
    equation.findRoots();

    return 0;
}
```

2.Free Fall Motion Simulation (Object-Oriented Approach in C++)

You are tasked with writing a C++ program that simulates the free fall motion of an object from a given height using object-oriented programming principles.

Problem Statement: Create a base class FallingObject that represents an object dropped from a certain height under the influence of gravity.

The class should have:

A constructor to initialize the initial height of the object.

A function heightAtTime(double time) to calculate the height of the object at a given time t using the formula:

$h(t) = h0 - \frac{1}{2}gt^2$

```
# include <iostream>

// Base class for a falling object
class FallingObject {
protected:
    double initialHeight;
    double gravity;
public:
    FallingObject(double h, double g = 9.81) : initialHeight(h), gravity(g) {}

    // Function to compute height at a given time
    double heightAtTime(double time) const {
        double height = initialHeight - 0.5 * gravity * time * time;
        return (height > 0) ? height : 0; // The object can't go below ground level
    }
};

int main() {
    double height;
    std::cout << "Enter initial height (meters): ";
    std::cin >> height;
```

```
    FallingObject object(height);

    double time;
    std::cout << "Enter time (seconds) to evaluate position: ";
    std::cin >> time;

    double currentHeight = object.heightAtTime(time);

    std::cout << "At time " << time << " seconds:" << std::endl;
    std::cout << "Height: " << currentHeight << " meters" << std::endl;

    return 0;
}
```

3.Simple Harmonic Motion (SHM) Simulator

Problem Statement: Implement a SHM class to calculate displacement, velocity, and acceleration at a given time.

Use the SHM equation:

$$x(t) = Acos(\omega t + \phi)$$

$$v(t) = -A\omega sin(\omega t + \phi)$$

$$a(t) = -A\omega^2 cos(\omega t + \phi)$$

where:

A = Amplitude

ω = sqrt(k/m) (Angular frequency)

φ = Phase angle

```
# include <iostream>
# include <cmath>

class SHM {
private:
    double amplitude, angularFrequency, phase;
public:
    SHM(double A, double w, double phi) : amplitude(A), angularFrequency(w), phase(phi) {}

    double displacement(double time) const {
        return amplitude * cos(angularFrequency * time + phase);
    }

    double velocity(double time) const {
```

```
        return -amplitude * angularFrequency * sin(angularFrequency * time + phase);
    }

    double acceleration(double time) const {
        return -amplitude * angularFrequency * angularFrequency * cos(angularFrequency * time + p
    }
};

int main() {
    double A, w, phi, t;
    std::cout << "Enter Amplitude (A): ";
    std::cin >> A;
    std::cout << "Enter Angular Frequency (w): ";
    std::cin >> w;
    std::cout << "Enter Phase Angle (phi): ";
    std::cin >> phi;
    std::cout << "Enter Time (t): ";
    std::cin >> t;

    SHM motion(A, w, phi);

    std::cout << "Displacement: " << motion.displacement(t) << " meters\n";
    std::cout << "Velocity: " << motion.velocity(t) << " m/s\n";
    std::cout << "Acceleration: " << motion.acceleration(t) << " m/s²\n";

    return 0;
}
```

4.

Double-click (or enter) to edit

```
Start coding or generate with AI.
```